Unit 13: Algorithms (Week 10)

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Part 1: Heuristics

Section 13.1

🤔 Some problems are too hard to solve exactly in any reasonable time.

Finding the optimal route through 50 cities would take longer than the age of the universe by brute force.

What do you do when the perfect answer is out of reach?

13.1 What Is a Heuristic?

A **heuristic** is a strategy that finds a *good enough* answer quickly, without guaranteeing it is the best possible answer.

	Exact algorithm	Heuristic
Result	Provably optimal	Good, possibly optimal
Speed	May be exponential	Usually fast
Guarantee	Always correct	No worst-case guarantee
Example	Dijkstra's	GPS "avoid highways"

Heuristics are used when the problem is NP-hard, the data is noisy, or speed matters more than perfection.

13.1 Heuristics in the Real World

Heuristics are everywhere in software you use daily:

- **GPS routing** — finds a good route fast; not always shortest
- **Chess engines** — evaluate positions with a heuristic score rather than searching all paths
- **Compilers** — heuristic register allocation (graph coloring approximation)
- **Search ranking** — Google's PageRank is a heuristic for "importance"
- **Spam filters** — heuristic rules flag likely spam without certainty
- **LLMs** — beam search finds likely output tokens heuristically

> A heuristic is not a bug — it is a deliberate engineering trade-off: trading a correctness guarantee for speed.

13.1 Three Levels of Approximation

Level	What you get	When to use
Exact	Optimal answer	Problem is tractable (polynomial)
Approximation algorithm	Answer within provable factor of optimal	NP-hard, bounded error acceptable
Heuristic	Empirically good answer	NP-hard, speed critical, no error bound needed

The **Travelling Salesman Problem (TSP)** — find the shortest tour through n cities — is NP-hard. No polynomial exact algorithm is known. But a nearest-neighbor heuristic runs in $O(n^2)$ and typically finds a tour within 20–25% of optimal.

Part 2: Greedy Algorithms

Section 13.3

🤔 You are making change for 67 cents using quarters, dimes, nickels, pennies.

You want to use as few coins as possible.

What strategy naturally comes to mind — and does it always give the fewest coins?

13.3 Greedy Algorithms

A **greedy algorithm** builds a solution one step at a time, always making the locally best choice at each step — without reconsidering earlier decisions.

```
Make change for 67 cents:
  Take largest coin ≤ remaining: 25¢  → remaining: 42¢
  Take largest coin ≤ remaining: 25¢  → remaining: 17¢
  Take largest coin ≤ remaining: 10¢  → remaining: 7¢
  Take largest coin ≤ remaining:  5¢  → remaining: 2¢
  Take largest coin ≤ remaining:  1¢  → remaining: 1¢
  Take largest coin ≤ remaining:  1¢  → remaining: 0¢

Result: 6 coins — optimal
```

13.3 When Greedy Works — and When It Doesn't

Greedy is correct when making the locally best choice never forecloses a globally better solution. This property is called the **greedy choice property**.

Greedy works:

- Coin change (standard denominations)
- Activity selection — pick the earliest-ending non-overlapping tasks
- Huffman coding — build optimal prefix-free codes
- Kruskal's / Prim's MST — always add the cheapest safe edge
- Dijkstra's shortest path — always expand the cheapest frontier

Greedy fails:

13.3 Activity Selection Problem

You have a set of tasks, each with a start and end time. You can only do one task at a time. **Maximize the number of tasks completed.**

```
Tasks:  A[1–4]  B[3–5]  C[0–6]  D[5–7]  E[3–9]  F[5–9]  G[6–10]

Greedy rule: always pick the task that ends soonest
  → A (ends 4), B (ends 5), D (ends 7), G (ends 10)  = 4 tasks
```

Why does earliest-end-time work? A task that ends sooner leaves more room for future tasks. Choosing any other task can only reduce the remaining opportunity.

> This is a clean example of the greedy choice property: the locally best choice (ends soonest now) is provably globally optimal.

13.3 Huffman Coding

Problem: encode characters as bit strings to minimize total message length. More frequent characters should get shorter codes.

Greedy strategy: repeatedly merge the two least-frequent symbols into a tree node.

```
Frequencies: A=45  B=13  C=12  D=16  E=9  F=5

Step 1: merge F(5) + E(9)  → FE(14)
Step 2: merge C(12) + B(13) → CB(25)
Step 3: merge FE(14) + D(16) → FED(30)
Step 4: merge CB(25) + FED(30) → CBFED(55)
Step 5: merge A(45) + CBFED(55) → root(100)
```

A = 0 (1 bit), B = 101, C = 100, D = 111, E = 1101, F = 1100. The most frequent character gets the shortest code.

Part 3: Greedy Algorithms in C++

Section 13.4

🤔 You have a knapsack that holds weight W. You have items with weights and values.

You want to maximize total value. You can take **fractions** of items (fractional knapsack).

Does a greedy strategy work here?

13.4 Fractional Knapsack

Take items in decreasing order of **value per unit weight** (value density). Take as much of each as fits.

```
sort(items.begin(), items.end(),
     [](auto& a, auto& b){ return a.vpw > b.vpw; });

double total = 0; int cap = W;
for (auto& item : items) {
    int take = min(item.weight, cap);
    total += take * item.vpw;
    cap -= take; if (!cap) break;
}
```

This greedy is **provably optimal** for the fractional knapsack — but fails for 0/1 knapsack (must take whole items or nothing).

13.4 Fractional Knapsack Trace

```
Items: A(w=10, v=60) B(w=20, v=100) C(w=30, v=120)  Capacity=50

Value densities:  A=6.0  B=5.0  C=4.0

Take all of A (10kg, $60) → remaining: 40kg
Take all of B (20kg, $100) → remaining: 20kg
Take 2/3 of C (20kg, $80) → remaining: 0kg

Total value: $240  ← optimal
```

For 0/1 knapsack with the same items: greedy takes A+B = $160. Optimal is B+C = $220. The greedy fails because you can't take a fraction of C to fill the gap.

13.4 Activity Selection in C++

```
sort(tasks.begin(), tasks.end(),
     [](auto& a, auto& b){ return a.end < b.end; });

int lastEnd = -1, count = 0;
for (auto& t : tasks) {
    if (t.start >= lastEnd) { lastEnd = t.end; count++; }
}
```

Sort by end time, then scan once. Any task that starts at or after the previous task ended is compatible — take it. Time: **O(n log n)** for the sort, O(n) for the scan.

Part 4: Dynamic Programming

Section 13.5

🤔 Compute the 40th Fibonacci number with this code:

```
int fib(int n) {
  if (n <= 1) return n;
return fib(n-1) + fib(n-2);
}
```

How many times is `fib(2)` called?

Is there a way to avoid recomputing the same subproblem over and over?

13.5 The Problem with Naive Recursion

`fib(40)` calls `fib(39)` and `fib(38)`. Each of those calls `fib(38)` and `fib(37)` — and `fib(38)` is computed **twice**. This doubles at every level.

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   │   ├── fib(2)  ← computed here
│   │   └── fib(1)
│   └── fib(2)      ← computed again!
└── fib(3)          ← entire subtree recomputed!
    ├── fib(2)      ← again!
    └── fib(1)
```

Total calls for `fib(n)`: $O(2^n)$. For n=40: over **1 billion** calls.

13.5 Dynamic Programming

Dynamic programming (DP) solves problems by breaking them into subproblems, solving each subproblem **once**, and storing the results.

Two requirements for DP to apply:

1. **Optimal substructure** — the optimal solution to the problem contains optimal solutions to its subproblems
2. **Overlapping subproblems** — the same subproblems are solved many times in naive recursion

If a problem has both properties, DP can turn an exponential algorithm into a polynomial one.

> DP is not a single algorithm — it is a *technique*. The insight is always the same: stop recomputing what you already know.

13.5 Memoization (Top-Down DP)

Store the result the first time a subproblem is solved. Return the stored result on every subsequent call.

```
int memo[100] = {};   // 0 means "not computed yet"

int fib(int n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}
```

`fib(n)` is now computed **exactly once** for each n. Total calls: O(n). Each call does O(1) work. Total time: **O(n)**.

13.5 Tabulation (Bottom-Up DP)

Fill a table from the smallest subproblems up to the answer. No recursion needed.

```
int fib(int n) {
    int dp[n+1];
    dp[0]=0; dp[1]=1;
    for (int i=2; i<=n; i++)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Tabulation is often preferred: no recursion stack, no risk of stack overflow, cache-friendly sequential access. Time: **O(n)**. Space: **O(n)** — or O(1) if you only keep the last two values.

13.5 0/1 Knapsack with DP

Greedy fails for 0/1 knapsack. DP solves it exactly.

State: `dp[i][w]` = maximum value using items 1..i with capacity w.

Recurrence:

```
dp[i][w] = dp[i-1][w]                              if item i doesn't fit
dp[i][w] = max(dp[i-1][w],
               dp[i-1][w - weight[i]] + val[i]) if item i fits
```

Each item is either taken or not — and the decision is recorded in the table so it is never recomputed.

13.5 Knapsack DP Trace

```
Items: A(w=2,v=3)  B(w=3,v=4)  C(w=4,v=5)   Capacity=5

      w=0  w=1  w=2  w=3  w=4  w=5
i=0    0    0    0    0    0    0
i=A    0    0    3    3    3    3
i=B    0    0    3    4    4    7   ← A+B fits in w=5
i=C    0    0    3    4    5    7

Answer: dp[3][5] = 7  (take A and B)
```

Time: **O(n × W)**. Space: **O(n × W)** — reducible to O(W) with a 1D table.

13.5 Longest Common Subsequence

Problem: given two strings, find the length of the longest subsequence common to both. (A subsequence need not be contiguous.)

```
s1 = "ABCBDAB"   s2 = "BDCAB"   LCS = "BCAB" or "BDAB" → length 4
```

Recurrence:

```
if s1[i] == s2[j]:  dp[i][j] = dp[i-1][j-1] + 1
else:               dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

LCS is used in `diff` (file comparison), DNA sequence alignment, and version control conflict detection.

13.5 DP vs. Greedy vs. Brute Force

Technique	Strategy	Optimal?	Time
Brute force	Try all possibilities	Always	Often exponential
Greedy	Best local choice	Sometimes	Usually O(n log n)
DP	Solve + store subproblems	Always (if applicable)	Polynomial
Heuristic	Good-enough rule	No guarantee	Fast

DP is strictly more powerful than greedy — but requires the problem to have optimal substructure. Greedy is simpler and faster when it applies.

> The most important skill in algorithm design is recognizing **which technique fits** the problem at hand.

Recognizing When to Use DP

Ask three questions:

1. **Can I define the answer in terms of smaller versions of the same problem?**
   → Optimal substructure exists
2. **Do those smaller problems repeat?**
   → Overlapping subproblems exist
3. **What is my "state"?**
   → The variables that uniquely identify a subproblem — this is your table dimension

```
Fibonacci:  state = n           (1D table)
Knapsack:   state = (item, capacity)   (2D table)
LCS:        state = (i, j) in s1, s2  (2D table)
```

The state definition is the hardest and most important step in any DP problem.

Unit 13 Summary

Concept	Key idea
Heuristic	Fast, good-enough answer; no optimality guarantee
Greedy	Locally best choice at each step; optimal when greedy choice property holds
DP (memoization)	Cache results top-down; O(n) instead of $O(2^n)$
DP (tabulation)	Fill table bottom-up; no recursion overhead
Optimal substructure	Solution built from optimal subsolutions
Overlapping subproblems	Same subproblem solved repeatedly in naive approach

That's all folks!